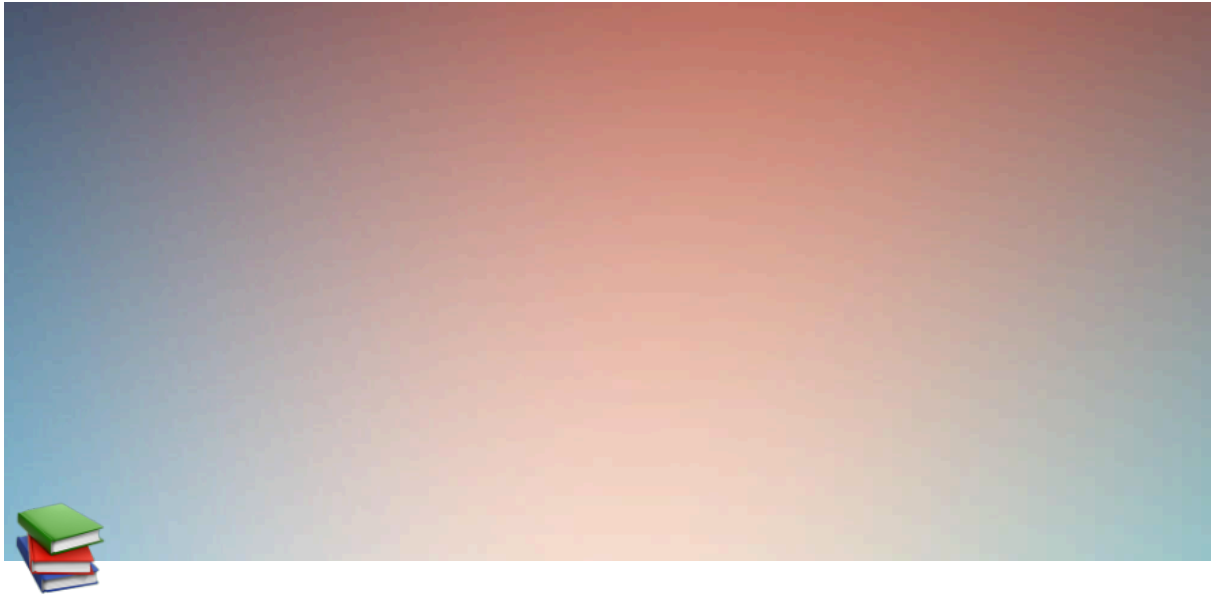




10주차 발표 요약본

5.0 토큰화 개요

#자연어와 자연어 처리

- 자연어: 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- 자연어 처리(NLP): 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

#토큰화

- 자연어 처리 모델을 만들기 위해 말뭉치를 토큰으로 나누어야 함
 - 말뭉치: 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어
 - 토큰: 개별 단어나 문장 부호와 같은 텍스트
 - 토큰화: 컴퓨터가 자연어를 이해할 수 있도록 토큰으로 나누는 과정, 토크나이저(텍스트 문자열을 토큰으로 나누는 알고리즘/소프트웨어)를 사용

5.1 단어 및 글자 토큰화

#단어 토큰화

- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
- 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용됨
- 가장 일반적인 토큰화 방법

#글자 토큰화

- 글자 단위로 문장을 나누는 방식
- 비교적 작은 단어 사전을 구축할 수 있다는 장점
-> 컴퓨터 자원의 효율적 사용 가능, 각 단어를 더 자주 학습할 수 있음
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용

5.2 형태소 토큰화

#형태소 토큰화

- 텍스트를 형태소 단위로 나누는 토큰화 방법
- 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류
- 교착어인 언어에서 중요하게 수행됨 (Ex. 한국어)
 - 형태소: 실제로 의미를 가지고 있는 최소의 단위
 - 자립 형태소: 스스로 의미를 가지고 있는 형태소, 단어를 이루는 기본 단위
 - 의존 형태소: 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 형태소

#형태소 어휘 사전

- 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중, 각 단어의 형태소 정보를 포함하는 사전
- 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있음
- 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 등의 정보를 함께 제공
- 품사 태깅: 텍스트 데이터를 형태소 분석하여 해당하는 품사를 태깅하는 작업
→ 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있음

#KoNLPy

- 한국어 자연어 처리를 위해 개발된 라이브러리
- 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공

- Okt, 꼬꼬마(Kkma), 코모란(Komoran) 등의 다양한 형태소 분석기 지원

#Okt

- 문장을 입력 받아 명사, 구, 형태소, 품사 등의 정보 추출하는 메서드 제공
- 대표적인 메서드
- 명사 추출(oka.nouns)
- 구문 추출(oka.phrases)
- 형태소 추출(oka.morphs)
- 품사 태깅(oka.pos)

#꼬꼬마(Kkma)

- 명사, 문장, 형태소, 품사 추출
- 구문 추출 기능을 제공하지 않음!

#KoNLPy의 라이브러리

- 동일한 메서드라도 분석기에 따라 다른 결과가 나타남
- 품사 태깅 역시 상이한 결과 제공
→ 시스템의 목적과 환경에 맞는 적절한 형태소 분석기를 사용해야 함

#NLTK

- 자연어 처리를 위해 개발된 라이브러리
- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공
- 주로 영어 자연어 처리를 위해 개발, 하지만 다양한 언어의 자연어 처리를 위한 데이터와 모델 제공
- 토큰화, 품사 태깅 작업을 위해 패키지나 모델을 다운로드 해야 함

#spaCy

- 사이썬을 기반으로 개발된 오픈 소스 라이브러리
- 자연어 처리를 위한 기능 제공
- 빠른 속도와 높은 정확도를 목표로 하는 머신 러닝 기반의 자연어 처리 라이브러리 (NLTK 라이브러리와 차이!)

5.3 하위 단어 토큰화

- 아이디어: 너무 미세(문자)도, 너무 거칠(단어)지도 않게 중간 단위로 분해
- 장점: OOV 감소, 어휘축소(메모리·연산 절감), 형태 변형(어미/접사) 대응
- 원칙: 자주 함께 나타나는 문자열을 묶어 서브워드 단위로 모델링

[토크나이저 학습 프로세스]

- 1.데이터 수집: 도메인 대표성/크기/클린
- 2.정규화: lowercasing, NFD/NFKC, 특수문자 규칙
- 3.프리토큰화: 공백/ByteLevel/Metaspace 등
- 4.모델 학습: BPE/WordPiece/Unigram, vocab 크기 결정
- 5.포스트프로세싱: [CLS]/[SEP] 삽입 등 템플릿 처리
- 6.평가: 토큰 길이, 커버리지, OOV, Perplexity 대리 지표
- 7.저장/배포: 버전 고정, 재현성/호환성 체크

[WordPiece 토크나이저 모델 학습 과정]

- 1.코퍼스 수집·정제(Korpora/사내 텍스트)
- 2.정규화 규칙 확정(NFKC, 숫자/기호 규칙)
- 3.프리토큰화(Whitespace/ByteLevel)
- 4.WordPiece 학습(어휘 크기, ## 접두사)
- 5.포스트프로세싱([CLS]/[SEP] 템플릿)
- 6.품질평가(토큰 길이/커버리지/OOV율)
- 7.버전관리 및 배포(JSON 저장)

06 임베딩

- 컴퓨터는 텍스트 자체를 이해할 수 없기 때문에 텍스트 벡터화 과정이 필요함
- **텍스트 벡터화(Text Vectorization)**
: 텍스트를 숫자로 변환하는 과정
- **원-핫 인코딩(One-Hot Encoding)**
: 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑 후 해당 색인 위치를 1로 표시, 나머지 위치는 모두 0으로 표시하는 방식
- **빈도 벡터화(Count Vectorization)**
: 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식

- 장점: 단어나 문장을 벡터 형태로 변환하기 쉽다
- 단점:
 - 벡터의 희소성(Sparsity)이 크다
 - 컴퓨팅 비용의 증가, 차원의 저주
 - 벡터가 텍스트의 의미를 반영하지 못한다
- **워드 임베딩**
: 단어를 고정된 길이의 실수 벡터로 표현하는 방법
- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론
- 대표 기법 : Word2Vec, fastText
- 단점 : 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다

⇒ **동적 임베딩(Dynamic Embedding) 기법** 사용

6.1 언어모델

- **언어모델(Language Model)**
: 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
- 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행
- 활용 분야: 자연어 처리 분야 (자동 번역, 음성 인식, 텍스트 요약 등)
- 단점 : 주어진 문장 뒤에 나올 수 있는 문장이 다양하기 때문에 완성된 문장 단위로 확률을 계산하는 것은 어려움

⇒ **자기회귀 언어 모델**: 하나의 토큰 단위로 예측

자기회귀 언어 모델(Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측
 - 조건부 확률: 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 예측하는 것
 - 여기에 조건부 확률의 연쇄법칙(Chain rule for conditional probability) 적용
 - 이를 통해 언어 모델은 문장 전체의 확률을 계산하고, 그것을 이용해 다음 단어를 예측

통계적 언어 모델 (Statistical Language Model)

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석
- 시퀀스에 대한 확률 분포를 추정 -> 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측

- 마르코프 체인(Markov Chain)을 이용해 구현됨
 - 마르코프 체인
 - : 빈도 기반의 조건부 확률 모델로, 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태 예측
 - 빈도 기반의 조건부 확률
 - 데이터 희소성(Data sparsity) 문제 발생

6.2 N-gram

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단어로 취급-> 특정 단어 시퀀스가 등장할 확률 추정
- 유니그램(N=1), 바이그램(N=2), 트라이그램(N=3), N-gram(N>=4)
- 활용 분야 : 관용적 표현 분석, 작은 규모의 데이터셋에서 연속된 문자열 패턴 분석, 자연어 처리 작업 및 문자열 패턴 분석

6.3 TF-IDF

- **TF-IDF(Term Frequency-Inverse Document Frequency)**
 - : 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법
- 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적 가중치
- Bow에 가중치를 부여하는 방법
 - **BoW (Bag-of-Words)**
 - : 문서나 문장을 단어의 집합으로 표현하는 방법
 - 문서/문장에 등장하는 단어의 중복을 허용해 빈도를 기록
- Bow 벡터화 예시
 - 모든 단어는 동일한 가중치를 갖는다
- **단어 빈도 (Term Frequency, TF)**
 - 문서 내에서 특정 단어의 빈도수를 나타내는 값
 - 해당 단어의 상대적인 중요도를 측정하는 데 사용
 - 높은 TF → 특정 문서에서 중요한 역할 / 전문 용어나 관용어

- **문서 빈도 (Document Frequency, DF)**

- 한 단어가 얼마나 많은 문서에 나타나는지
- 높은 DF → 일반적으로 널리 사용되는 단어 / 중요도 낮음
- 낮은 DF → 특정 문맥에서만 사용되는 단어 / 중요도 높음

- **역문서 빈도(Inverse DF, IDF)**

- 문서 내에서 특정 단어가 얼마나 중요한지
- 전체 문서 수를 문서 빈도로 나눈 다음 로그 취한 값
- 높은 IDF → 일반적이고 중요하지 않은 단어

- **TF-IDF**

- 문서 빈도와 역문서 빈도를 곱한 값
- 관사/관용어 등의 가중치 낮아짐

- **주요 메서드 및 속성**

- `toarray()`: TF-IDF 결과를 넘파이 배열 형태로 변환
- 배열 형태: (문서 수, 단어 수)의 2차원 형태
- `vocabulary_`속성
 - 키(key): 단어
 - 값(value): 단어의 색인(index)

- **TF-IDF의 한계점**

- 문맥과 단어의 순서를 고려하지 않음
-> 문장 생성이나 문장 순서가 중요한 작업에 부적합
- 단어의 의미를 반영하지 않음
-> TF-IDF 벡터는 중요도만 나타내며, 단어 간 의미적 유사성을 표현하지 못함